Guion de la exposición

Tópicos en Aprendizaje Automático (UNAJ) — Marcos David Kantor — Análisis crítico de Trockman y Kolter (2023)

Duración objetivo: 18 min 50 s de exposición y debate hasta completar 30 min. Regla: el título de cada diapositiva ya es el mensaje; decirlo primero y después mostrar la prueba.

1. Portada [0:20]

Buenas tardes. Soy Marcos Kantor. Elegí analizar el artículo «Patches Are All You Need?», de Asher Trockman y Zico Kolter, publicado en Transactions on Machine Learning Research en 2023. Además del análisis, repliqué parte de sus experimentos.

2. Contenido de la exposición [0:50]

Sigo el orden que pide la consigna: el artículo, la introducción, el modelo, la metodología, los resultados y las conclusiones de los autores. Después muestro mi réplica y cierro con una reflexión. La idea en una frase: una red convolucional muy simple, que recibe la imagen en parches como un ViT, compite con los transformers. En mi computadora, sin GPU, llegó a 82,7 % en CIFAR-10.

3. El artículo seleccionado [1:40]

Es un artículo de Carnegie Mellon y del Bosch Center for AI. Cumple la consigna: es reciente, propone un modelo, lo compara con varios algoritmos (ResNet, ViT, DeiT, MLP-Mixer) y tiene resultados en dos conjuntos de datos. Uno de ellos es CIFAR-10, que es público, y eso me permitió replicarlo.

Lo ubico en la Unidad 4, redes convolucionales. En la réplica sumé un modelo de cada una de las otras unidades para poder comparar. Lo conocí porque estaba en la bibliografía de Sistemas de Inteligencia Artificial del ITBA, que curso en paralelo.

4. El problema: el ViT cambió dos cosas a la vez [2:50]

Hasta 2020 la visión por computadora era de las CNN, que son piramidales: bajan la resolución y suben los canales. El Vision Transformer cambió dos cosas a la vez: usa atención y, como la atención es cuadrática, corta la imagen en parches de 16 por 16. Además es isotrópico: misma resolución en todas las capas.

La pregunta del artículo es cuál de las dos cosas explica la mejora. Si son los parches, podemos quedarnos con la convolución, que aprende con menos datos, sin pagar el costo de la atención. Eso importa en dominios con pocos datos, como las imágenes médicas de mi tesis.

5. El ConvMixer [4:20]

El modelo tiene tres partes. Primero, una convolución con paso igual al núcleo, que corta la imagen en parches y proyecta cada uno a h canales. Después, d bloques iguales, cada uno con una convolución depthwise de núcleo grande, un filtro por canal, que mezcla información espacial, con una conexión

residual, y una convolución uno por uno que mezcla los canales. Al final, pooling global y una capa lineal.

Tiene cuatro hiperparámetros: ancho, profundidad, parche y núcleo. La cantidad de parámetros tiene una fórmula cerrada, la ecuación 4, que yo verifiqué contra PyTorch. Es aprendizaje supervisado con entropía cruzada y AdamW.

6. Cómo encaja con la Unidad 4 [5:10]

Todo lo que usa es de la Unidad 4: convoluciones, separables, SGD, BatchNorm y residuales. Lo que no vimos en el curso son los modelos con los que compara: el ViT, que usa atención; el MLP-Mixer, que mezcla con dos MLP, y Swin. Lo importante: el ConvMixer es un MLP-Mixer con convoluciones en lugar de MLP. Si rinde parecido, lo que importa es lo que tienen en común.

7. Metodología de los autores [6:10]

Usan ImageNet, con un millón de imágenes, y CIFAR-10. No hay selección de características: los filtros se aprenden. El preprocesamiento es sobre todo aumentación de datos, y es muy fuerte. Entrenan con AdamW y una tasa triangular, entre 90 y 300 épocas, unos 9 días en 10 GPU.

Dicen que no buscaron hiperparámetros en ImageNet, pero en CIFAR-10 sí probaron a mano ancho, profundidad, parche, núcleo y weight decay, y no informan un conjunto de validación. Desde la Unidad 1 eso es un problema: elegir mirando el test infla el resultado.

8. Resultados del artículo en ImageNet [7:10]

El ConvMixer de 52 millones de parámetros llega a 81,4 %, por encima de la ResNet-152 entrenada con la misma receta y cerca de DeiT. El de 21 millones iguala a la ResNet-152 con un tercio de los parámetros. Pero es entre 4 y 10 veces más lento, y sólo la ResNet se entrenó en igualdad de condiciones.

9. Ablaciones en CIFAR-10 [8:00]

En CIFAR-10 muestran que el núcleo grande ayuda (93,6 % con k=3 y 95,9 % con k=9) y que los parches chicos rinden más. Lo que más me llamó la atención es la tabla de aumentación: sacar el escalado aleatorio cuesta casi 10 puntos, mucho más que cualquier cambio de arquitectura. La receta de entrenamiento es parte del resultado.

10. Conclusiones y limitaciones del artículo [9:00]

Los autores concluyen que el diseño isotrópico con parches es una plantilla potente por sí misma. Mi lectura crítica: no hay un experimento factorial que aísle los parches, las recetas de los modelos comparados son distintas, hay una sola corrida por modelo, sin barras de error, y el costo es enorme.

11. Réplica: el experimento del Apéndice B, a escala de CPU [10:10]

Repliqué el experimento de CIFAR-10, bajando los datos de la página oficial de Toronto. Mantuve la arquitectura, el optimizador, la tasa triangular, el recorte de gradiente y el escalado aleatorio, y comparé con una ResNet con la misma receta. Lo que cambié fue por cómputo: 10 épocas en lugar de 200 y un modelo más chico, porque en mi computadora el modelo del paper tardaba tres horas y media por época.

Y fui más estricto que el artículo: separé 5000 imágenes de validación y miré el test una sola vez.

12. Resultado: CNN muy arriba; ConvMixer y ResNet empatan [11:30]

El ConvMixer llegó a 82,7 % y la ResNet a 83,0 %: empatan. Abajo están los modelos del curso: el perceptrón y la seudoinversa rondan el 40 %; la ELM llega a 47,3 %; el MLP, a 57,4 %. El MLP tiene 16 veces más parámetros que el ConvMixer y queda 25 puntos abajo: lo que importa es el sesgo inductivo de la convolución, no la cantidad de parámetros.

13. Curvas de validación [12:10]

Con 10 épocas todos los modelos siguen aprendiendo. La curva se aplana al final sólo porque la tasa de aprendizaje llega a cero. Ahí está la mayor parte de la distancia con el 95 % del paper.

14. Ablaciones frente a la Tabla 4 [13:30]

El efecto del parche se reproduce: con p=4 pierdo 4,0 puntos. El del núcleo no: con k=3 gano 2,0 puntos. Mi explicación es que con pocas épocas converge antes el modelo más simple, y los propios autores dicen que el efecto del núcleo grande podría depender de entrenar más. Con una sola semilla, lo tomo como indicio, no como conclusión. ConvMixer y ResNet difieren en 0,3 puntos, que está dentro del ruido.

15. El costo sí se reproduce [14:10]

La ResNet es 3,7 veces más rápida en inferencia. En entrenamiento, 32 minutos contra 110. Esto coincide con el artículo: la convolución depthwise de núcleo grande está poco optimizada.

16. Réplica en GPU (Google Colab) [15:10]

Para comprobar que la diferencia era de cómputo y no un error mío, corrí el modelo en una GPU gratuita de Google Colab con la receta oficial de los autores para CIFAR-10. Llegó a 94,5 %, frente al 95,2 % del paper para esa configuración. O sea, la implementación es correcta y la brecha es de escala.

17. CPU frente a GPU [16:10]

Esta tabla me parece lo más interesante para discutir. En mi computadora cada época tardó 11 minutos; en la GPU, 56 segundos. En un tiempo parecido, la GPU entrenó un modelo casi tres veces más grande, con muchas más épocas y con la aumentación completa, y ganó 12 puntos con el mismo código.

La pregunta que dejo para el debate: ¿cuánto de «el mejor modelo» es en realidad «el que más cómputo recibió»? Es la misma crítica que le hago al artículo cuando compara recetas distintas, y tiene un costado ético: sin GPU sólo se puede verificar una versión reducida.

18. Reflexión crítica [17:30]

Lo elegí porque lo había trabajado en el ITBA, porque se podía replicar sin un cluster y porque en mi tesis tengo que elegir entre CNN y transformers con pocos datos. Podría usarse en segmentación

médica, microscopía o señales como el ECG. Para entenderlo necesité las unidades 3 y 4 y, para criticarlo, la Unidad 1.

En lo ético destaco el costo de cómputo, que concentra la investigación en quien tiene GPU; la reproducibilidad; los sesgos de ImageNet, y el doble uso para diagnóstico o vigilancia.

19. Conclusiones [18:30]

El artículo tiene una idea fuerte con evidencia parcial. Mi réplica confirma que la implementación funciona, que el ConvMixer es lento y que el parche importa; no confirma, con poco entrenamiento, la ventaja del núcleo grande ni la ventaja sobre la ResNet. Replicar no es copiar un número: es poder explicar cada diferencia.

20. Reproducibilidad y cierre [18:50]

Todo el código, las semillas y los resultados están en el repositorio y el notebook. Gracias; quedo abierto a preguntas.

Preguntas probables y respuestas

- **¿Por qué tu número no da igual que el del paper?**
  Por cómputo: 10 épocas en lugar de 200, un modelo 3 veces más chico y sin RandAugment. En GPU, con la receta de los autores, llegué a 94,5 % frente a 95,2 %.
- **¿Por qué el núcleo chico te dio mejor?**
  Con pocas épocas converge antes el modelo con menos parámetros y un sesgo más local. Es una sola semilla. Los autores sugieren que el efecto del núcleo grande depende de entrenar más o de regularizar más.
- **¿Qué es una convolución depthwise?**
  Una convolución con un filtro por canal (groups igual a la cantidad de canales). No mezcla canales; eso lo hace la pointwise de 1x1 que viene después.
- **¿Por qué el ConvMixer es lento si tiene pocos parámetros?**
  Porque mantiene una resolución interna alta en todas las capas y la convolución depthwise de núcleo grande tiene poca reutilización de datos y está poco optimizada. Los FLOPs no son el único costo.
- **¿Cómo elegiste los hiperparámetros sin mirar el test?**
  Con 5000 imágenes de validación. Para la seudoinversa usé validación cruzada de 5 pliegues. El test se evaluó una sola vez.
- **¿Por qué la ELM rinde menos que el MLP?**
  Porque sus pesos de entrada son aleatorios: sólo aprende la capa de salida. En imágenes crudas eso no alcanza para capturar la estructura espacial. Su ventaja es que se entrena en minutos con una SVD.
- **¿Por qué la seudoinversa necesitó un λ tan grande?**
  Porque los píxeles vecinos están muy correlacionados y X^T X está mal condicionada. Tikhonov estabiliza la solución, y la validación cruzada eligió $\lambda = 10000$.
- **¿Qué es el sesgo inductivo?**
  Los supuestos que trae el modelo antes de ver datos. En la convolución: localidad y pesos compartidos, que en imágenes valen más que los parámetros.

- **¿Usaste IA para hacer el trabajo?**
  Usé un asistente para escribir código y redactar, y lo declaro. Verifiqué cada número: los parámetros contra la ecuación 4, el test una sola vez, todo generado desde los resultados guardados.